

Relazione per progetto di
Programmazione a Oggetti
Battaglia Navale

Marco Penolazzi
Angelica Sartori
Tetyana Volosohzar

1° settembre 2026

Capitolo 1 Analisi	3
1.1 Descrizione e requisiti	3
Requisiti funzionali	3
Requisiti non funzionali.....	5
1.2 Modello del Dominio.....	5
Partita 6	
Giocatore.....	6
Griglia e coordinate.....	6
Nave 6	
Colpo 7	
Colpo doppio.....	7
Colpo sequenziale	7
Sonar 7	
Turno ed esito	7
Capitolo 2 Design	10
2.1 Architettura	10
2.2 Design dettagliato	11
2.2.1 Design dettagliato - Marco Penolazzi	11
2.2.2 Design dettagliato - Angelica Sartori.....	20
2.2.3 Design dettagliato - Tetyana Volosozhar	29
Capitolo 3 Sviluppo	35
3.1 Testing automatizzato	35
3.2 Note di sviluppo	35
3.2.1 Descrizione di alcune funzionalità utilizzate nella realizzazione della GUI - Tetyana Volosozhar	35
3.2.2 Descrizione di alcune funzionalità utilizzate nella realizzazione di dinamiche, colpi ed eventi speciali - Angelica Sartori.....	36
3.2.3 Descrizione di alcune funzionalità avanzate nella realizzazione del Model - Marco Penolazzi	37
Capitolo 4 Commenti finali	38
4.1 Autovalutazione e lavori futuri	38
4.1.1 Autovalutazione - Tetyana Volosozhar.....	38
4.1.2 Autovalutazione - Angelica Sartori.....	38
4.1.3 Autovalutazione - Marco Penolazzi	39
Appendice A	41
Guida utente	41
Appendice B.....	42
Esercitazioni di laboratorio.....	42
B.O.1 marco.penolazzi@studio.unibo.it	42

Capitolo 1 Analisi

1.1 Descrizione e requisiti

Il progetto consiste nello sviluppo di una versione modificata ed espansa del gioco "Battaglia Navale". Prima dell'inizio della partita, ciascun giocatore posiziona la propria flotta su una griglia quadrata di dimensione 10x10. Le colonne della griglia sono identificate dalle lettere comprese tra A e J, mentre le righe sono identificate dai numeri compresi tra 1 e 10.

Ogni giocatore dispone della seguente flotta:

- un'ammiraglia di lunghezza 5;
- una nave da battaglia di lunghezza 4;
- due incrociatori di lunghezza 3;
- un sottomarino invisibile di lunghezza 3;
- due cacciatorpediniere di lunghezza 2;
- una nave da ricognizione di 3 celle disposte a L;
- facoltativamente, una nave corazzata lineare di lunghezza 3.

Le navi con forma lineare possono essere disposte orizzontalmente o verticalmente all'interno della griglia di gioco. La nave da ricognizione supporta 4 differenti rotazioni (0°, 90°, 180°, 270°). Durante il posizionamento, nessuna nave può uscire dai limiti della griglia o sovrapporsi a un'altra nave. È consentito collocare due navi in celle adiacenti.

Dopo aver terminato il posizionamento di entrambe le flotte dei due giocatori, inizia la fase di combattimento. I giocatori effettuano le proprie azioni a turno senza poter vedere la posizione delle navi avversarie non ancora individuate.

Requisiti funzionali

Il sistema consente ad ogni giocatore di:

- posizionare le navi della flotta prima dell'inizio della partita;
- scegliere le coordinate in cui effettuare l'attacco;

- visualizzare l'esito di ogni azione;
- utilizzare le abilità speciali quando disponibili;
- conoscere a quale giocatore appartiene il turno corrente;
- conoscere la conclusione della partita con il relativo vincitore;

Il colpo singolo interessa una coordinata della griglia avversaria. Se nella coordinata selezionata è presente una nave, il colpo risulta riuscito e il giocatore conserva il turno. Se viene colpita l'ultima cella integra di una nave, la nave viene considerata affondata. Se nella coordinata non è presente alcuna nave, il colpo risulta mancato e il turno passa all'avversario.

Se la coordinata che viene scelta è già stata colpita o è oltre i limiti della griglia, il giocatore deve selezionare una nuova coordinata e il turno rimane al giocatore corrente.

Il colpo doppio è una abilità speciale che permette di attaccare due coordinate differenti durante la stessa azione. Inizialmente l'abilità non è disponibile e viene caricata dopo tre coordinate colpite con successo con singoli colpi. I risultati ottenuti tramite il colpo doppio non contribuiscono alla sua ricarica. Dopo un utilizzo valido il colpo doppio torna scarico. Se almeno uno dei due colpi va a segno il turno continua altrimenti se entrambi i colpi mancano il turno passa all'avversario. I colpi mancati non azzerano la ricarica del colpo doppio.

Il colpo sequenziale è un'abilità speciale che permette di colpire tre coordinate purché siano consecutive e sulla stessa linea orizzontale o verticale. Se le coordinate selezionate non rispettano questa disposizione, il colpo viene rifiutato. L'abilità è inizialmente disponibile per il giocatore e può essere utilizzata una sola volta durante la partita. Dopo un utilizzo valido, non è più disponibile e non può essere nuovamente utilizzata.

Ogni giocatore possiede un singolo utilizzo del sonar. Il sonar analizza un'area 3x3 della griglia di gioco avversaria comunicando la presenza di parti di navi rilevabili non ancora colpite in quell'area restituendo un numero compreso tra 0 e 9. Il sottomarino invisibile è immune al sonar e non viene incluso nel risultato della scansione ma, come le altre navi presenti nella griglia, è vulnerabile a tutte le modalità di tiro. La scansione fornisce solo informazioni senza danneggiare le imbarcazioni o modificare la griglia. Dopo l'utilizzo del sonar il turno passa all'avversario.

La partita termina quando tutte le navi di uno dei due giocatori sono state affondate. Il giocatore che ha ancora almeno una nave non affondata viene

dichiarato vincitore mentre l'altro viene dichiarato sconfitto.

Il sistema deve fornire riscontri per differenziare i seguenti eventi:

- colpo riuscito;
- nave affondata;
- colpo mancato;
- colpo assorbito;
- coordinata già selezionata;
- coordinata non valida;
- conclusione del turno;
- vittoria;
- sconfitta.

Requisiti non funzionali

L'applicazione deve presentare le informazioni di gioco in modo chiaro, permettendo ai giocatori di distinguere facilmente le celle non ancora selezionate, i colpi riusciti e i colpi mancati.

Le posizioni delle navi di un giocatore non devono essere mostrate all'avversario prima che vengano colpite. Le azioni non valide non devono alterare la configurazione della griglia, lo stato delle navi, la carica delle abilità o il giocatore corrente.

Le regole del gioco devono produrre risultati deterministici a parità di stato e input. La casualità degli eventi deve essere isolata attraverso una sorgente iniettabile, così da poter utilizzare seed noti e ottenere test riproducibili. L'applicazione deve anche gestire correttamente tutte le fasi della partita, dal posizionamento iniziale fino alla dichiarazione del vincitore.

1.2 Modello del Dominio

Il dominio dell'applicazione è formato dagli elementi necessari per rappresentare il gioco della battaglia navale, dalla disposizione iniziale delle flotte fino alla scelta del vincitore.

Le principali entità individuate sono:

Partita

Rappresenta l'intero svolgimento del gioco e coinvolge due giocatori. Mantiene lo stato corrente del gioco, distingue la fase di posizionamento dalla fase di combattimento e stabilisce quale giocatore possiede il turno. Inizialmente entrambi i giocatori devono completare il posizionamento della propria flotta.

Successivamente la partita gestisce l'alternarsi dei turni e termina quando uno dei due giocatori ha l'intera flotta affondata.

Giocatore

È uno dei due partecipanti alla partita. Ogni giocatore ha:

- una griglia personale;
- una flotta classica composta da un totale di 8 navi oppure quando l'opzione globale è abilitata, una flotta di nove navi comprendente anche la nave corazzata;
- lo stato di caricamento del colpo doppio;
- un singolo utilizzo del sonar.
- la disponibilità del tiro sequenziale utilizzabile una sola volta.

Il giocatore attivo può effettuare un colpo singolo oppure, quando disponibili, utilizzare il sonar, il colpo doppio o il colpo sequenziale.

Griglia e coordinate

Rappresenta l'area nella quale vengono posizionate le navi di un giocatore. E' composta da 10 righe e 10 colonne. Ogni cella è identificata da una coordinata con colonne A-J e righe 1-10. Una cella può essere in vari stati:

- acqua;
- cella ignota all'avversario;
- cella occupata da nave visibile;
- sezione colpita;
- colpo mancato;
- sezione appartenente a una nave affondata;

Nave

Appartiene alla flotta di un solo giocatore e occupa un insieme di coordinate nella sua griglia.

Il numero e la disposizione delle coordinate dipendono dal tipo di nave. Le navi lineari possono essere posizionate verticalmente o orizzontalmente mentre la nave da ricognizione occupa tre celle disposte a forma di L.

Una nave può trovarsi nei seguenti stati:

- integra, se non è ancora stata colpita in nessun punto;
- danneggiata, se almeno un suo punto è stato colpito ma una parte è ancora integra
- affondata, quando tutte le sue parti sono state colpite.

Il sottomarino invisibile non viene rilevato dal sonar ma può essere colpito e

affondato normalmente.

Colpo

È una azione compiuta dal giocatore attivo contro la griglia di gioco dell'avversario.

Il colpo singolo interessa una singola coordinata e può produrre vari risultati:

- colpo riuscito;
- nave affondata;
- colpo mancato;
- colpo assorbito dalla corazza.

La selezione di una coordinata esterna alla griglia o già colpita rende l'azione invalida. In questo caso lo stato della partita, il turno e le abilità del giocatore non vengono modificate.

Il risultato del colpo determina anche la gestione del turno. Colpire o affondare una nave permette al giocatore attivo di continuare il proprio turno mentre un colpo mancato o colpo assorbito determina il passaggio del turno all'avversario.

Colpo doppio

Il colpo doppio è una abilità associata al giocatore che permette di attaccare due coordinate differenti nella stessa azione.

L'abilità viene caricata dopo aver colpito con successo tre coordinate tramite colpi singoli. Quando viene utilizzata, l'abilità si scarica. Se almeno uno dei due attacchi colpisce una nave, il giocatore conserva il turno; se almeno uno dei due risultati è HIT o SUNK il giocatore conserva il turno; se nessun risultato rappresenta un danno effettivo, il turno passa all'avversario. Anche ARMOR_ABSORBED, come MISS, non mantiene il turno.

Colpo sequenziale

Il colpo sequenziale è un'abilità associata al giocatore che permette di attaccare tre coordinate appartenenti alla stessa linea, in orizzontale o verticale, nella stessa azione.

L'abilità può essere usata una volta sola per l'intera durata di una partita. Se, durante l'uso, almeno uno dei colpi risulta in un colpo riuscito, il giocatore conserva il turno; se nessun bersaglio produce HIT o SUNK, il turno passa all'avversario.

Sonar

È un'abilità utilizzabile una sola volta da ciascun giocatore che analizza un'area di 3x3 celle della griglia avversaria fornendo informazioni sulla presenza di parti di navi rilevabili.

Le parti appartenenti al sottomarino invisibile non vengono considerate nel risultato della scansione. L'utilizzo valido del sonar conclude sempre il turno del giocatore.

Turno ed esito

Il turno identifica il giocatore autorizzato a compiere una azione e dipende

da quale azione è stata eseguita in precedenza e il relativo risultato. La partita termina quando l'intera flotta di un giocatore risulta affondata. Il giocatore che possiede almeno una nave non affondata nella sua flotta è il vincitore della partita.

Oltre alle entità principali, il dominio comprende valori e stati necessari per descrivere in modo ben definito la partita. Ogni cella è individuata da una coordinata mentre giocatori e navi possiedono identità stabili. Un'azione offensiva può essere un colpo normale, un colpo doppio o un colpo sequenziale e, per ogni bersaglio valido, viene prodotto un esito che distingue un colpo mancato, un colpo riuscito, un colpo assorbito e l'affondamento di una nave. L'esito di un colpo è distinto dallo stato mostrato da una cella della griglia: il primo descrive un evento appena avvenuto mentre il secondo rappresenta la situazione osservabile della cella. La partita si suddivide in preparazione, svolgimento e conclusione. Le richieste che non rispettano le regole, come una coordinata esterna alla griglia o già selezionata, sono considerate azioni invalide e non costituiscono un normale esito del colpo.

Le entità descritte e i loro rapporti sono rappresentati in modo sintetico all'interno nella Figura 1.1. La partita coinvolge due giocatori, ciascuno dotato di una griglia e della propria flotta. Le coordinate rappresentano le celle occupate, i bersagli dei colpi e il centro del sonar. Un'azione di tiro può comprendere uno, due oppure tre bersagli e produce un `TurnResult` con gli esiti relativi alle singole coordinate. Il sonar produce invece un `SonarResult` con il numero di sezioni rilevate.

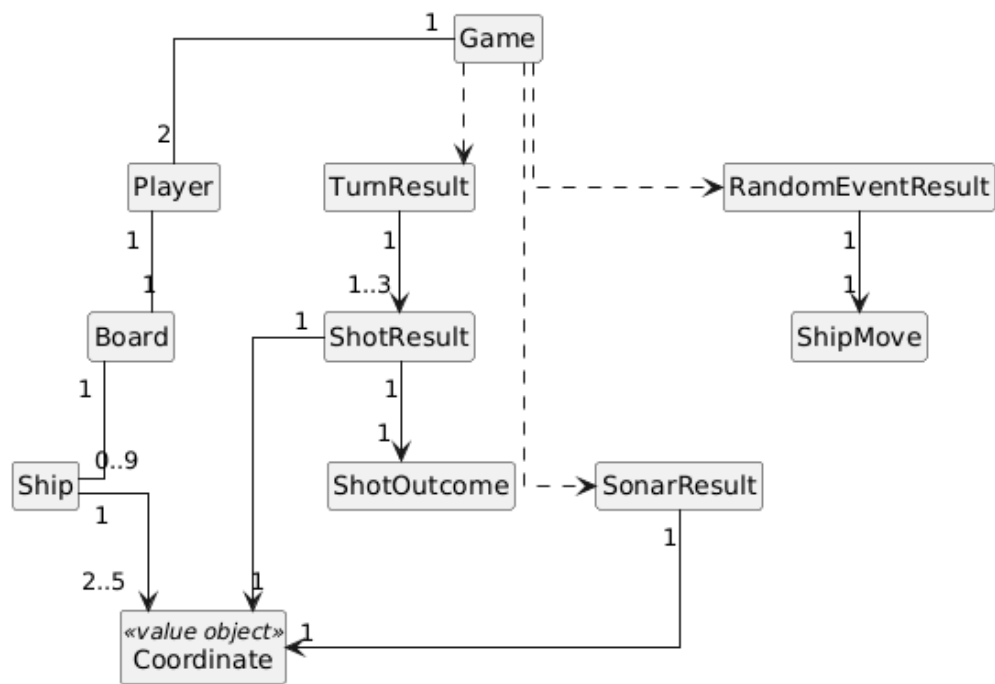


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali e i loro rapporti

Capitolo 2 Design

2.1 Architettura

L'applicazione segue il pattern architetturale Model-View-Controller. Il Model contiene entità, regole e stato della partita e non importa classi Swing. Game costituisce il principale contratto per coordinare una partita, mentre Board espone le operazioni relative alla singola flotta.

GameImpl e BoardImpl implementano tali contratti. La View è descritta dall'interfaccia BattleshipView e riceve dal Controller oggetti di presentazione immutabili, fra cui PlacementViewState, GameViewState e HandoffViewState; BattleshipFrame è la sua implementazione Swing.

Le azioni dell'utente non richiamano metodi concreti del Controller: la View le comunica tramite BattleshipViewObserver. BattleshipController implementa tale interfaccia, conserva lo stato della sessione, interroga o modifica il Model e ordina alla View quale schermata mostrare. BattleshipApp costruisce BattleshipFrame, BattleshipController, StandardGameFactory e la sorgente casuale; quindi, collega la View al proprio Observer. In questo modo il Model è riutilizzabile indipendentemente dalla GUI e il Controller dipende dal contratto BattleshipView, non da JFrame.

La sostituzione di Swing richiederebbe una nuova implementazione della View e dei testi di presentazione, senza modificare le regole del Model.

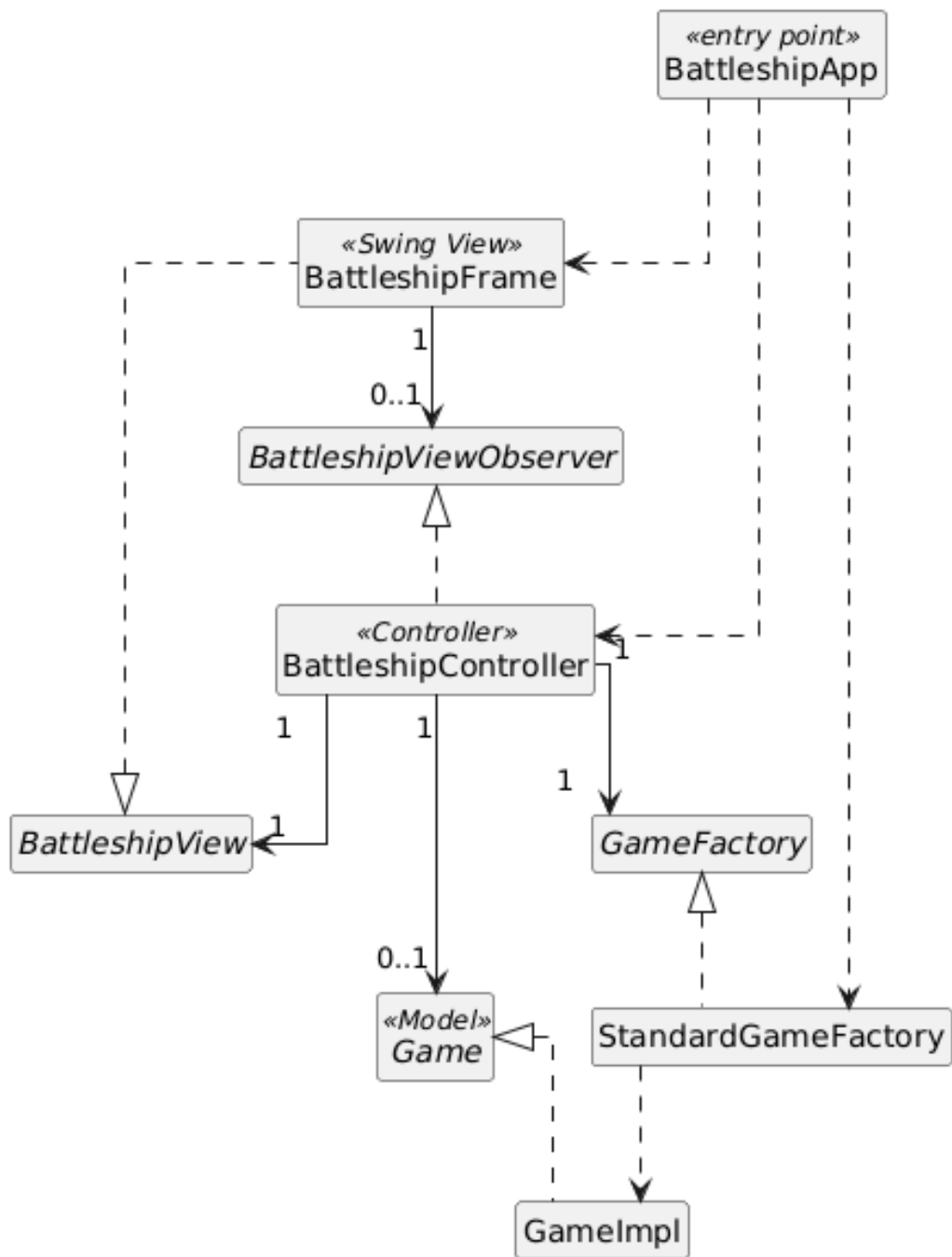


Figura 2.1. Struttura architetturale MVC e direzione delle dipendenze tra Model, View e Controller

2.2 Design dettagliato

2.2.1 Design dettagliato - Marco Penolazzi

2.2.1.1 Valori di dominio e gestione violazioni

- Problema

I primi concetti del Model richiedono di rappresentare coordinate, identità, modalità di attacco, stati ed esiti senza l'utilizzo di coppie di interi anonimi o stringhe modificabili liberamente. Usando tali rappresentazioni si potrebbero avere valori privi di significato, confronti fragili e duplicazione dei controlli. È inoltre necessario distinguere le varie violazioni delle regole senza legare il Model ai messaggi o agli elementi dell'interfaccia grafica.

- Soluzione

Le classi `Coordinate` e `ShipId` sono state realizzate come value object immutabili. `Coordinate` rifiuta componenti negative e mette a disposizione `isInside` per verificare l'appartenenza a una griglia con dimensione nota; `ShipId` rifiuta i valori nulli o privi di contenuto. L'immutabilità evita che una posizione o una identità possano cambiare dopo la costruzione e il confronto del contenuto rende questi oggetti adatti per essere usati come valori del dominio.

Gli insiemi finiti di alternative sono rappresentati tramite enum. `PlayerId` identifica i due partecipanti e permette di ottenere l'avversario con `other`; `ShotKind` distingue `NORMAL`, `DOUBLE` e `SEQUENTIAL`; `ShotOutcome` rappresenta `MISS`, `HIT`, `SUNK` e `ARMOR_ABSORBED`; `CellState` descrive la situazione osservabile di una cella; `GamePhase` distingue `READY`, `IN_PROGRESS` e `FINISHED`. Separare `ShotOutcome` e `CellState` evita di confondere l'evento appena prodotto da un colpo con la successiva rappresentazione della griglia.

`ShotResult` associa una `Coordinate` a uno `ShotOutcome` e impedisce che i componenti siano nulli. Il metodo `isHit` restituisce true soltanto per `HIT` e `SUNK`. `ARMOR_ABSORBED` indica infatti che l'impatto è avvenuto ma non ha prodotto danno: non conserva il turno e non carica il colpo doppio. Questa interpretazione viene definita una sola volta nel Model e riutilizzata da `GameImpl` e dal livello di presentazione.

Le richieste che violano le regole vengono classificate da `RuleViolation`, che include cause come coordinata esterna, bersaglio già colpito, duplicazione, quantità errata, flotta incompleta o abilità non disponibile. `GameRuleException` è una eccezione di dominio unchecked che conserva sia il messaggio diagnostico sia la `RuleViolation`. Il Model comunica quindi il motivo semantico del rifiuto senza decidere come mostrarlo; `SwingText` traduce successivamente tale valore in una frase destinata all'utente.

- Conseguenza

La soluzione introduce diversi tipi di piccole dimensioni, ma riduce l'uso di primitivi e stringhe non controllate, rende espliciti i contratti e garantisce gli invarianti al momento della costruzione. Le violazioni risultano confrontabili nei

test automatici e possono essere presentate da una GUI diversa senza modificare il Model. In questa parte non è stato applicato un pattern GoF: sono stati usati value object, record, enum e una eccezione semantica per rendere il dominio più sicuro e comprensibile

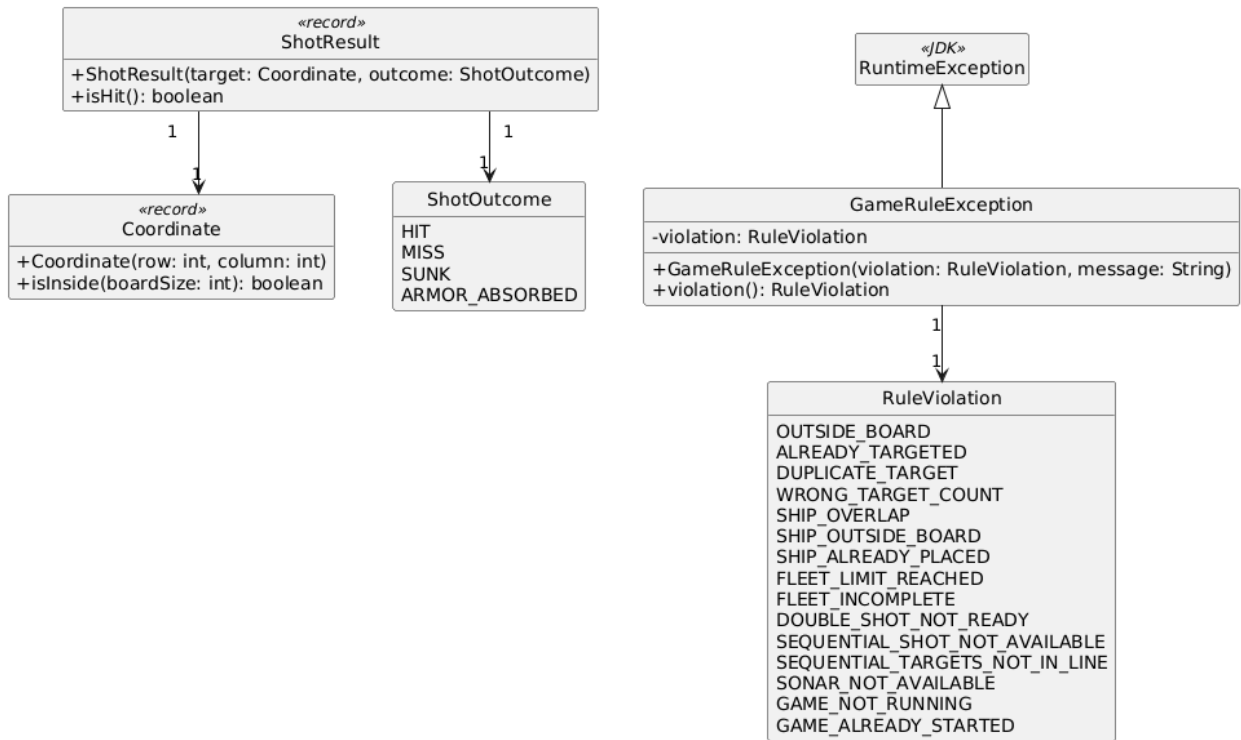


Figura 2.2. Value object, esiti del tiro e classificazione delle violazioni delle regole

2.2.1.2 Modellazione delle navi e visibilità

- Problema

La flotta comprende navi con lunghezze, forme e rotazioni differenti e può includere una corazzata opzionale. Il Model deve impedire identificatori duplicati, forme incoerenti, posizionamenti esterni e sovrapposizioni. La griglia deve inoltre applicare un'azione con più bersagli interamente oppure rifiutarla senza effetti parziali, proteggere la posizione delle navi avversarie e permettere al sonar di ignorare il sottomarino invisibile senza renderlo immune ai colpi.

- Soluzione

ShipType contiene soltanto le caratteristiche intrinseche di ogni tipo, cioè lunghezza e **ShipShape**. Le quantità non appartengono più all'enum: **FleetRules** è un record immutabile che normalizza una quantità per ogni **ShipType**, esegue

una copia difensiva e offre le configurazioni `classic` e `withArmoredShip`. La flotta classica richiede otto navi, mentre la seconda configurazione aggiunge una corazzata. `BoardImpl` riceve `FleetRules` nel costruttore, usa lo stesso oggetto per limitare il piazzamento e considera completa la flotta soltanto quando la quantità di ogni tipo coincide esattamente con la regola. `GameImpl` rifiuta due `Board` costruite con regole differenti.

`Ship` è una classe final il cui stato è protetto tramite incapsulamento. Il metodo statico `place` costruisce una nave a partire da `ShipId`, `ShipType`, origine e `Rotation`, ruota gli offset della forma per quarti di giro e normalizza le coordinate rispetto all'origine. Il costruttore privato verifica lunghezza e unicità delle celle e ne conserva una copia non modificabile. Per la `RECON`, che ha forma a L, le rotazioni 0°, 90°, 180° e 270° producono quattro insiemi distinti. `Ship.place` è una static factory utile a garantire un oggetto valido.

`Board` descrive le operazioni di dominio relative a piazzamento, attacco, completezza, affondamento, sonar, movimento e snapshot. `BoardImpl` usa `Map<Coordinate, Ship>` per individuare rapidamente la nave presente in una cella, `List<Ship>` per attraversare la flotta e `Set<Coordinate>` per registrare senza duplicati le celle realmente risolte. `fireAt` copia la lista, rifiuta liste vuote e coordinate duplicate e valida tutti i bersagli prima di applicare il primo. Se una coordinata di un colpo doppio o sequenziale è invalida, nessuna delle altre viene modificata.

Per la nave corazzata la protezione è associata a ogni singola sezione. Il primo impatto sulla sezione produce `ARMOR_ABSORBED`, non viene registrato come danno e la coordinata resta selezionabile; il secondo impatto produce `HIT` o `SUNK` e diventa un bersaglio già risolto. Durante un evento casuale `BoardImpl` considera soltanto navi non affondate e destinazioni interne, libere, non colpite e completamente disgiunte dalla vecchia posizione. `Ship.relocate` trasferisce per indice relativo lo stato di danno e di corazza alla nuova forma. Le vecchie sezioni già colpite restano osservabili come `MISS`; quelle mai colpite tornano acqua per il proprietario e ignote per l'avversario.

`BoardSnapshot` è una proiezione immutabile della griglia e conserva copie non modificabili degli stati e dei tipi visibili. `BoardImpl` delega la visibilità a `ShipVisibilityPolicy`, che rappresenta una Strategy: `OwnerOnlyVisibilityPolicy` mostra una nave integra soltanto al proprietario. `InvisibleSubmarinePolicy` implementa la stessa interfaccia e avvolge una policy di base, delegandole la normale visibilità ma escludendo il sottomarino invisibile dal sonar. Si tratta di un Decorator perché conserva l'astrazione e aggiunge una responsabilità senza modificare la policy decorata.

- Alternativa

Le quantità avrebbero potuto restare in `ShipType` e le regole speciali essere

gestite con condizioni dentro `BoardImpl`. Questa soluzione avrebbe però mescolato caratteristiche intrinseche e configurazione della partita, duplicato i controlli fra GUI e Model e richiesto modifiche alla griglia per ogni diversa visibilità. `FleetRules`, `Strategy` e `Decorator` localizzano invece gli aspetti variabili e consentono test indipendenti dalla rappresentazione interna.

- **Conseguenza**

La soluzione protegge gli invarianti di nave e flotta, mantiene atomiche le azioni multiple e impedisce alla View di accedere allo stato mutabile della `Board`. Il costo è l'introduzione di più oggetti immutabili e copie difensive, accettabile per una griglia 10×10. La configurazione della corazzata rimane esplicita e identica per entrambi i giocatori; nuove regole di visibilità possono essere aggiunte senza modificare `BoardImpl`.

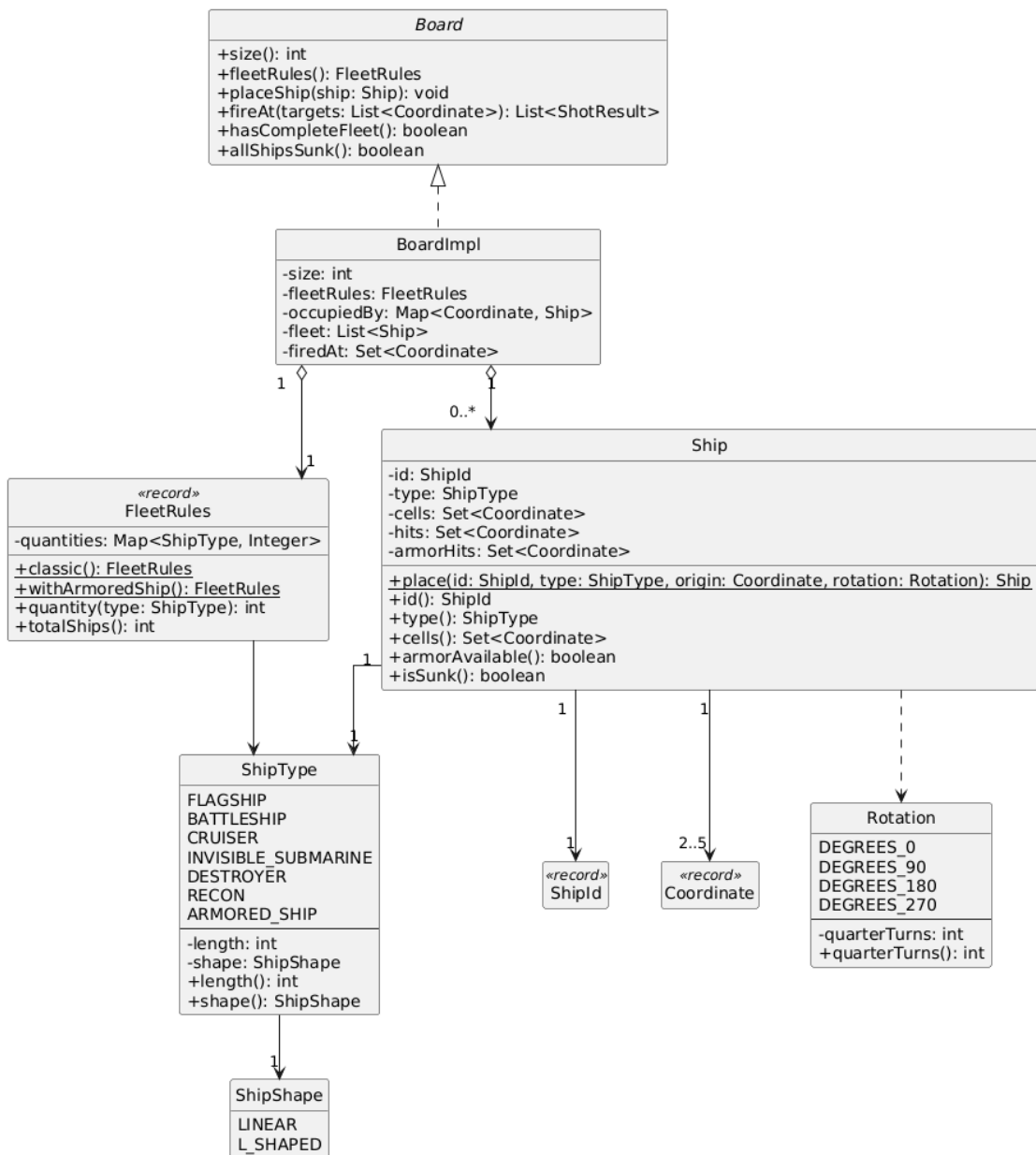


Figura 2.3. Relazioni tra nave, configurazione flotta e griglia di gioco

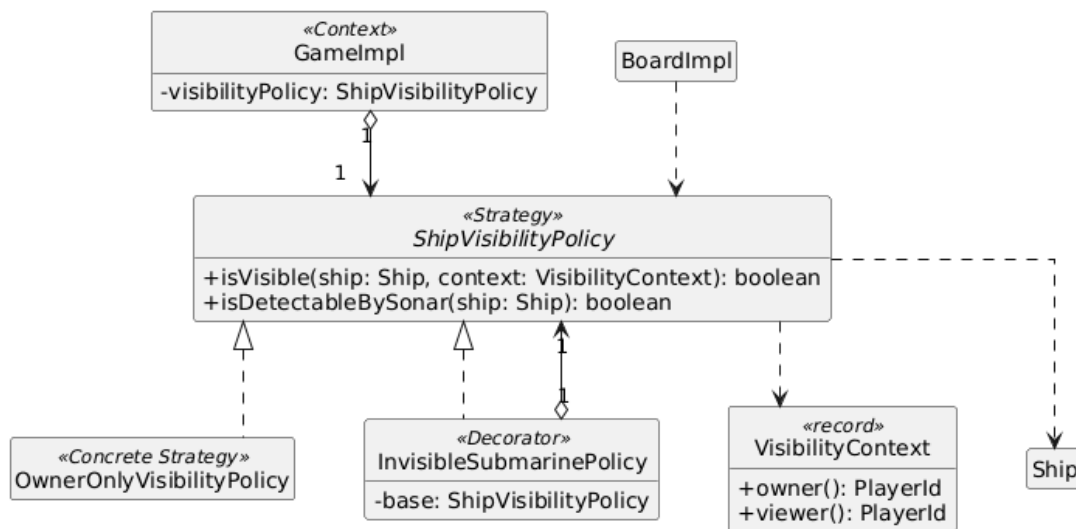


Figura 2.4. Applicazione pattern Strategy e Decorator alla visibilità delle navi

2.2.1.3 Strategie di tiro e gestione della carica

- Problema

Il colpo normale, doppio e sequenziale condividono controlli comuni ma richiedono rispettivamente uno, due e tre bersagli; il sequenziale impone inoltre tre celle consecutive orizzontali o verticali. Duplicare l'intero flusso oppure accumulare condizioni in GameImpl renderebbe fragile l'ordine delle violazioni. Occorre anche conservare nel Model la carica del doppio colpo e impedire alla View di modificarla.

- Soluzione

ShotStrategy rappresenta il pattern Strategy e mantiene il solo contratto execute. AbstractShotStrategy implementa uno scheletro final: controlla Board e lista, crea una copia immutabile, verifica quantità e duplicati, richiama validateSpecificTargets e infine delega a Board.fireAt. NormalShotStrategy e DoubleShotStrategy specificano soltanto la cardinalità; SequentialShotStrategy verifica tre coordinate consecutive, accettandole anche in ordine inverso dopo l'ordinamento. AbstractShotStrategy.execute è un Template Method reale perché definisce una sequenza stabile e lascia a una operazione protetta la parte variabile.

GameImpl riceve una mappa fra ShotKind e ShotStrategy, la copia in un EnumMap e richiede esplicitamente una strategia per ogni valore dell'enum. StandardGameFactory è un servizio di composizione che costruisce la configurazione standard con NormalShotStrategy, DoubleShotStrategy,

SequentialShotStrategy e la policy di visibilità decorata.

DoubleShotAbility incapsula un progresso compreso tra zero e tre. Soltanto un colpo NORMAL con almeno un esito HIT o SUNK aumenta il progresso; MISS e ARMOR_ABSORBED non lo incrementano. Quando la carica raggiunge tre il colpo DOUBLE è disponibile; il consumo riporta il progresso a zero, quindi l'abilità può essere caricata nuovamente. DoubleShotStatus esporta una fotografia immutabile e valida la coerenza fra progress, requiredHits e ready. Il colpo SEQUENTIAL è invece disponibile una sola volta per ciascun giocatore e viene consumato soltanto dopo una esecuzione valida.

- Conseguenza

Strategy separa gli algoritmi di tiro dal coordinamento della partita e il Template Method elimina la duplicazione rendendo deterministico l'ordine delle violazioni. La dipendenza esplicita consente di sostituire strategie nei test e fa fallire subito una configurazione incompleta. Il costo è una piccola gerarchia di classi, compensato dall'estensione localizzata: una nuova modalità di tiro richiede una strategia e una voce di configurazione, mentre GameImpl conserva soltanto le regole di disponibilità e consumo.

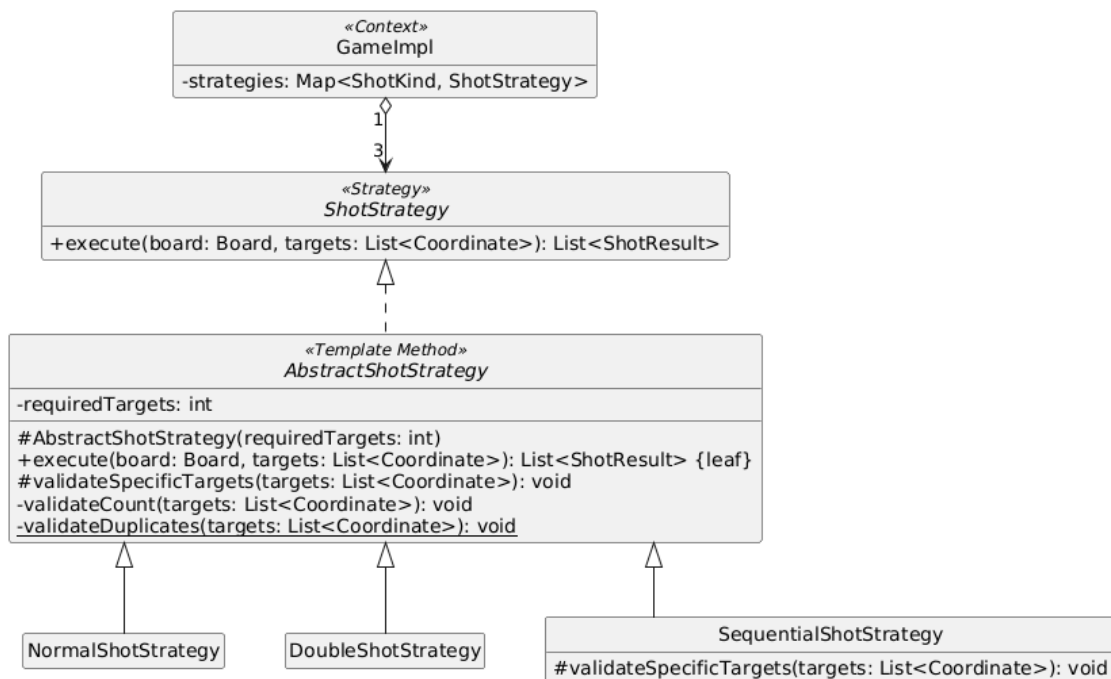


Figura 2.5. Applicazione dei pattern Strategy e Template Method alle modalità di tiro

2.2.1.4 Coordinamento della partita e transizioni di stato

- Problema

La partita deve accettare azioni diverse in base alla fase corrente, individuare il giocatore attivo, conservare il turno dopo almeno un `HIT` o `SUNK`, alternarlo dopo `MISS`, `ARMOR_ABSORBED` o sonar e terminare quando viene affondata l'ultima nave. Un'azione non valida non deve modificare griglia, carica, abilità, fase o giocatore corrente. Queste decisioni appartengono al Model e non possono essere demandate alla View.

- Soluzione

`Game` è il contratto del motore; `GameImpl` coordina due Player, le strategie, la `ShipVisibilityPolicy` e una `RandomGenerator` ricevuti dall'esterno. Il costruttore rifiuta giocatori duplicati, `Board` con `FleetRules` differenti e mappe prive di una `ShotStrategy`. Lo stato iniziale è `READY`; `start` è accettato una sola volta e passa a `IN_PROGRESS` soltanto se entrambi i giocatori possiedono una flotta completa. `GamePhase` rappresenta una macchina a stati tramite enum.

`playTurn` salva l'identità del giocatore agente, verifica la disponibilità dell'abilità e delega l'azione alla strategia associata. Soltanto dopo una esecuzione riuscita aggiorna carica o consumo. La presenza di almeno un `ShotResult` per cui `isHit` restituisce true mantiene il turno; in caso contrario il turno passa all'avversario. Se tutte le navi bersaglio sono affondate, la fase diventa `FINISHED`, `winner` contiene il giocatore agente e non esiste un `nextPlayer`. Il sonar valido viene consumato una sola volta e conclude sempre il turno.

`TurnResult` copia gli esiti e verifica la coerenza fra `phase`, `nextPlayer` e `winner`. `GameSnapshot` raccoglie per un viewer la fase, il giocatore corrente, il vincitore opzionale, le due `BoardSnapshot` filtrate e lo stato delle abilità. `triggerRandomEvent` è disponibile soltanto durante `IN_PROGRESS`, usa la `RandomGenerator` iniettata e tenta di spostare una nave non affondata dei due giocatori; restituisce `Optional.empty` quando nessun movimento è possibile. L'iniezione della sorgente casuale rende i test riproducibili.

- Alternativa

Turno, vittoria e abilità avrebbero potuto essere gestiti dal Controller. In tal caso il Model sarebbe diventato una semplice struttura dati e la stessa partita avrebbe potuto comportarsi in modo diverso in una GUI o in un test. Centralizzare le regole in `GameImpl` mantiene invece un unico punto di verità e consente di eseguire una partita completa senza Swing.

- Conseguenza

L'ordine validazione-esecuzione-aggiornamento rende le azioni rifiutate prive di effetti e permette test deterministici del motore. L'iniezione di strategie, policy e casualità riduce l'accoppiamento e `StandardGameFactory` concentra la

composizione standard. Il costo è una responsabilità significativa in `GameImpl`, ma limitata all'avanzamento della partita; la presentazione, la privacy del dispositivo e la temporizzazione degli eventi restano nel Controller e nella View.

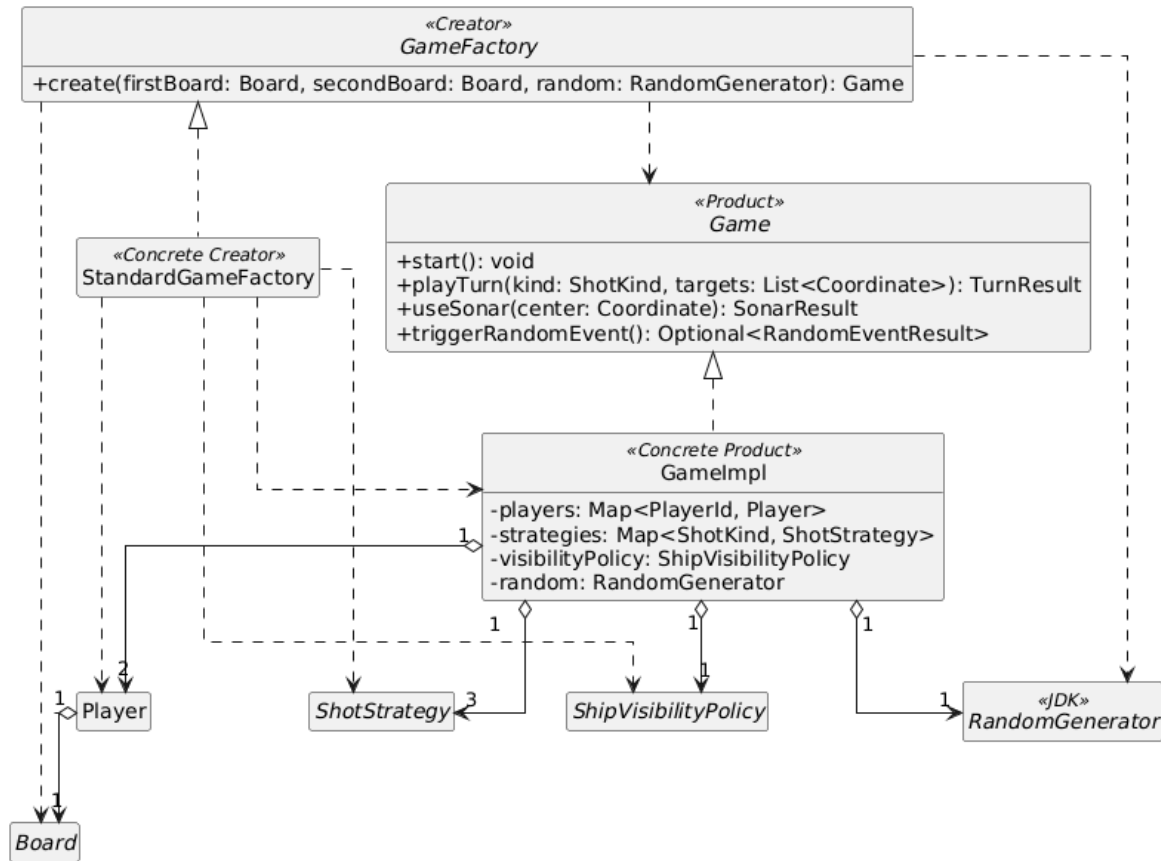


Figura 2.6. Coordinamento della partita e creazione della configurazione standard

2.2.2 Design dettagliato - Angelica Sartori

2.2.2.1 Gestione della nave corazzata

- Problema

Tra le meccaniche speciali del gioco è prevista una nave corazzata, caratterizzata dalla capacità di assorbire un colpo prima che una propria cella venga danneggiata.

L'introduzione di questo nuovo tipo di nave richiede di distinguere un colpo assorbito da un normale `HIT`, poiché nel primo caso la nave non subisce ancora un danno e il colpo non deve essere considerato riuscito dalle altre meccaniche di gioco.

Una prima implementazione della corazza utilizzava un unico valore booleano associato all'intera nave. In questo modo, però, il primo colpo ricevuto

consumava la protezione per tutta la nave, mentre il comportamento desiderato richiede che ogni sua cella disponga della propria protezione e debba quindi essere colpita due volte: il primo colpo viene assorbito dalla corazza e il secondo produce il danno effettivo.

L'aggiunta della nave corazzata pone un problema di configurazione della flotta. La scelta deve valere nello stesso modo per entrambi i giocatori: con `FleetRules.classic` la nave non è richiesta, mentre con `FleetRules.withArmoredShip` diventa parte della flotta completa di entrambe le Board.

- Soluzione

La nave corazzata è rappresentata dal valore `ARMORED_SHIP` di `ShipType`, che contiene soltanto lunghezza e forma. L'opzionalità non è memorizzata nell'enum: `FleetRules` definisce in modo immutabile la quantità di ogni tipo e offre le configurazioni `classic` e `withArmoredShip`. Entrambe le Board ricevono la stessa configurazione e `BoardImpl.hasCompleteFleet` richiede esattamente le quantità previste; `GameImpl` rifiuta Board costruite con regole differenti.

Per rappresentare il comportamento della corazza è stato introdotto `HitEffect`, che distingue internamente i due possibili effetti di un colpo che raggiunge una nave: `ABSORBED`, quando il colpo viene assorbito dalla protezione, e `DAMAGED`, quando provoca un danno effettivo. A livello del risultato del tiro è stato inoltre aggiunto `ARMOR_ABSORBED` a `ShotOutcome`. In `BoardImpl` l'esito restituito da `Ship.registerHit` viene utilizzato per produrre il corrispondente `ShotResult`.

`ShotResult.isHit` è stato adeguato affinché `ARMOR_ABSORBED`, come `MISS`, non venga considerato un colpo riuscito. In questo modo un tiro che ha soltanto consumato la protezione della nave non viene interpretato dalle altre meccaniche del Model come un vero danno.

La rappresentazione interna dell'armatura è stata successivamente modificata passando da un singolo valore `armorAvailable`, relativo all'intera nave, a un insieme di coordinate `armorHits`. L'insieme registra le celle della nave sulle quali la corazza ha già assorbito il primo colpo. In `Ship.registerHit`, quando una coordinata di `ARMORED_SHIP` non è ancora presente nell'insieme, essa viene aggiunta ad `armorHits` e il metodo restituisce `ABSORBED`; un secondo colpo sulla stessa coordinata può invece essere registrato nell'insieme dei danni effettivi.

La modifica ha richiesto anche un adeguamento della validazione dei bersagli in `BoardImpl`. Una coordinata sulla quale la corazza ha assorbito il colpo deve poter essere selezionata nuovamente, mentre una cella già realmente danneggiata deve continuare a produrre la violazione `ALREADY_TARGETED`. La rappresentazione osservabile della griglia distingue allo stesso modo una cella

soltanto colpita sulla corazza da una cella effettivamente danneggiata.

Infine, poiché le navi possono essere spostate dalle altre meccaniche del gioco, è stato previsto che anche lo stato relativo ai colpi assorbiti dalla corazza venga trasferito durante la rilocalizzazione, insieme allo stato dei danni già ricevuti, evitando che lo spostamento ripristini completamente la protezione della nave.

La prima implementazione della meccanica è stata accompagnata da test dedicati in `SpecialMechanicsTest`, utilizzati per verificare l'assorbimento del colpo e la possibilità di colpire nuovamente una coordinata protetta dalla corazza.

- **Conseguenza**

La nave corazzata dispone di uno stato distinto rispetto alle navi normali e può rappresentare separatamente l'assorbimento della corazza e il danno effettivo. Il passaggio da uno stato globale a uno stato associato alle coordinate permette di modellare la protezione per ogni singola cella della nave, rendendo il comportamento più aderente alla regola secondo cui ciascuna sezione deve essere colpita due volte.

La distinzione tra `ARMOR_ABSORBED` e `HIT` evita che un impatto privo di danno venga interpretato come un successo dalle altre meccaniche. `FleetRules` permette inoltre di aggiungere la nave speciale a entrambe le flotte senza mescolare la configurazione della partita con le caratteristiche intrinseche di `ShipType`.

La soluzione aumenta lo stato da mantenere per la nave corazzata rispetto a una semplice variabile booleana, ma permette di rappresentarne in modo esplicito il comportamento e di mantenere separate la protezione della corazza, il danno effettivo e la composizione obbligatoria della flotta.

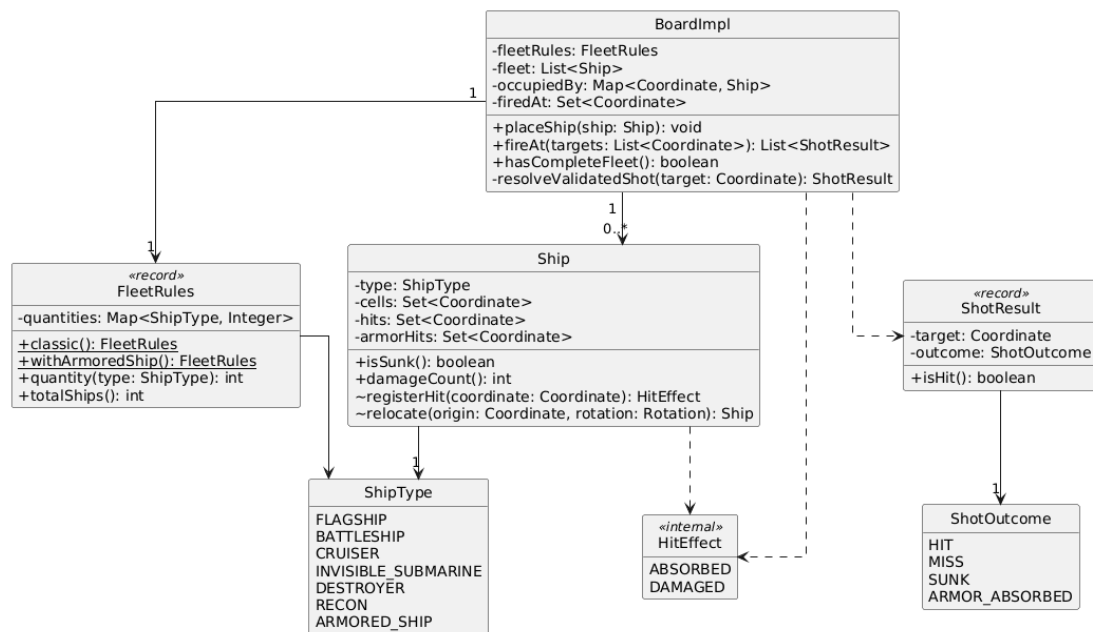


Figura 2.7. Stato della corazza per sezione e risultato dei colpi

2.2.2.2 Gestione del tiro sequenziale e dei colpi speciali

- Problema

Il sistema di gestione dei colpi prevedeva inizialmente il colpo normale e il colpo doppio, ma era necessario introdurre una nuova modalità di attacco speciale denominata tiro sequenziale. Questa azione permette al giocatore di colpire tre celle consecutive disposte sulla stessa riga oppure sulla stessa colonna.

L'introduzione del nuovo tiro richiede di controllare non soltanto il numero di bersagli selezionati, ma anche la loro disposizione geometrica. Tre coordinate qualsiasi non devono quindi essere considerate valide: devono appartenere alla stessa linea ed essere consecutive.

Il tiro sequenziale deve inoltre essere utilizzabile una sola volta da ciascun giocatore durante la partita. È quindi necessario mantenere nel Model lo stato di disponibilità dell'abilità, impedirne un secondo utilizzo e rendere tale informazione disponibile anche attraverso lo stato osservabile della partita.

- Soluzione

Il nuovo tipo di attacco è stato aggiunto all'enum `ShotKind` mediante il valore `SEQUENTIAL`, integrandolo nelle modalità di tiro già supportate dal Model. La relativa logica è stata implementata nella classe `SequentialShotStrategy`, che implementa l'interfaccia `ShotStrategy` già utilizzata per rappresentare le diverse modalità di attacco.

`SequentialShotStrategy` estende `AbstractShotStrategy`, che riceve nel costruttore il numero esatto di bersagli e applica in un metodo `execute final` le validazioni comuni: lista non nulla, cardinalità, duplicati e successiva chiamata a `Board.fireAt`. La strategia sequenziale ridefinisce soltanto `validateSpecificTargets` per controllare la disposizione geometrica delle tre coordinate.

Le coordinate ricevute vengono ordinate per riga e colonna e successivamente viene verificata una delle due configurazioni ammesse. Nel caso orizzontale, le tre coordinate devono appartenere alla stessa riga e avere colonne consecutive; nel caso verticale devono invece appartenere alla stessa colonna e avere righe consecutive. Se sono presenti tre bersagli ma non rispettano una delle due disposizioni, viene sollevata una `GameRuleException` associata alla nuova `RuleViolation.SEQUENTIAL_TARGETS_NOT_IN_LINE`.

La disponibilità del tiro sequenziale viene mantenuta all'interno di `Player` attraverso il valore booleano `sequentialShotAvailable`, inizialmente impostato a `true`. Il metodo `canUse` è stato adeguato affinché distingua le condizioni di utilizzo dei diversi valori di `ShotKind`: il colpo normale è sempre disponibile, il colpo doppio continua a dipendere dal proprio stato preesistente, mentre il tiro sequenziale può essere utilizzato soltanto finché `sequentialShotAvailable` rimane vero.

Dopo l'utilizzo del tiro sequenziale, `consumeSequentialShot` rende l'abilità non più disponibile. Un eventuale nuovo tentativo produce la violazione `SEQUENTIAL_SHOT_NOT_AVAILABLE`, evitando che la regola di utilizzo singolo debba essere controllata da componenti esterni al `Model`.

La nuova modalità è stata inoltre integrata nella gestione del turno di `GameImpl`. Prima dell'esecuzione viene verificata la disponibilità del tipo di tiro scelto; successivamente l'attacco viene delegato alla strategia associata a `ShotKind`. Se il tipo utilizzato è `SEQUENTIAL`, l'abilità viene consumata dopo l'esecuzione. La disponibilità residua viene infine inclusa in `GameSnapshot`, permettendo ai livelli superiori dell'applicazione di conoscere lo stato dell'abilità senza accedere direttamente all'oggetto `Player`.

La correttezza della strategia è stata verificata tramite un test dedicato in `SpecialMechanicsTest`. Il test controlla sia un caso valido, formato da tre celle consecutive sulla stessa linea, sia un caso non valido, verificando che venga restituita la corrispondente `RuleViolation`.

- **Conseguenza**

Il tiro sequenziale viene integrato nel sistema dei colpi senza introdurre la relativa logica direttamente nella gestione generale della partita. La specifica regola geometrica rimane infatti incapsulata in `SequentialShotStrategy`, mentre `GameImpl` continua a coordinare genericamente l'esecuzione delle diverse

strategie in base al relativo **ShotKind**.

La disponibilità dell'abilità viene mantenuta nel Model come parte dello stato del giocatore, impedendo utilizzi multipli e rendendo superfluo affidare tale controllo alla View o al Controller. L'estensione di **GameSnapshot** permette inoltre di esporre questa informazione mantenendo separato lo stato interno dalla sua rappresentazione.

La soluzione riutilizza l'astrazione **ShotStrategy** già presente nel progetto, aggiungendo soltanto il comportamento specifico richiesto dal nuovo tiro. In questo modo l'introduzione della nuova modalità non modifica la logica interna delle strategie di tiro esistenti e mantiene separate la validazione generale dei bersagli dalla regola specifica di consecutività.

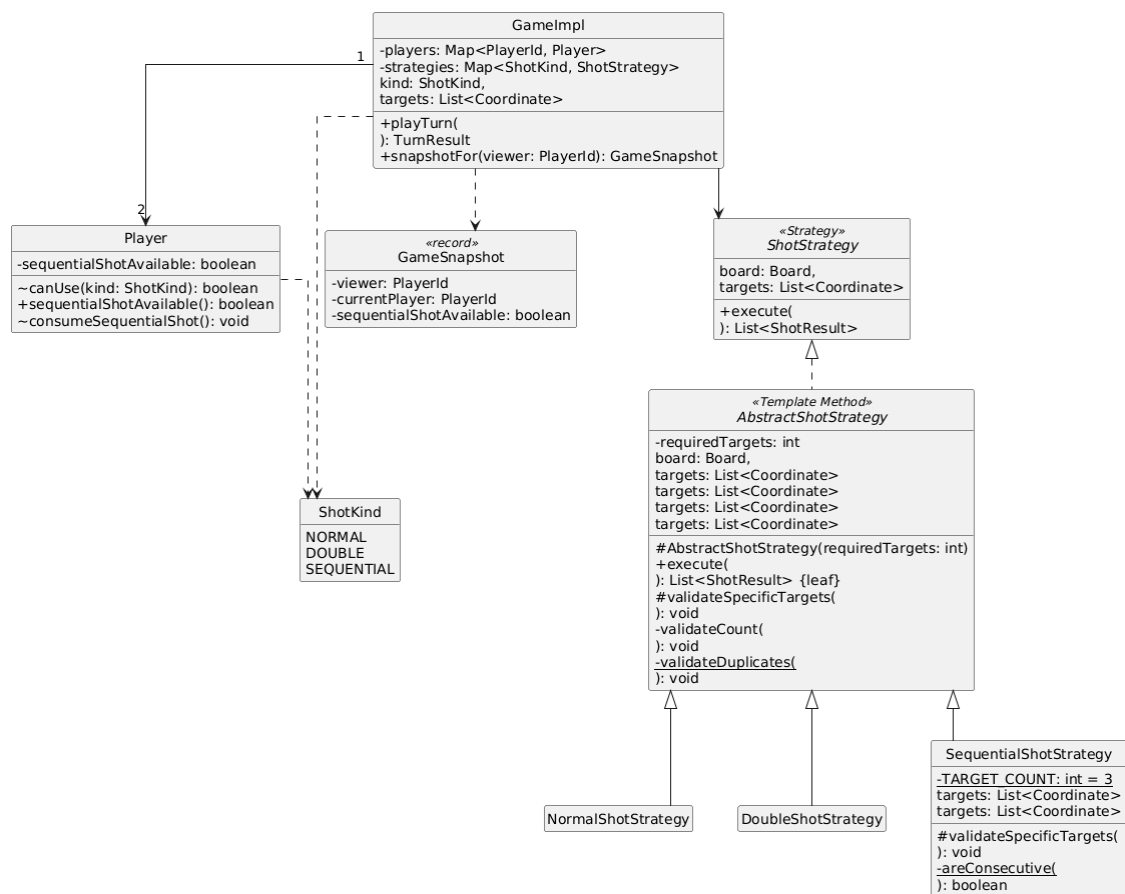


Figura 2.8. Tiro sequenziale e disponibilità dell'abilità

2.2.2.3 Gestione dell'evento casuale e dello spostamento delle navi

- Problema

Tra le meccaniche speciali della partita è previsto un evento casuale capace di

modificare temporaneamente la disposizione della flotta spostando una nave ancora attiva. L'evento deve poter essere applicato durante una partita in corso senza compromettere la validità della griglia e senza alterare lo stato già raggiunto dalle navi.

Lo spostamento non può quindi essere effettuato scegliendo una posizione arbitraria. La nuova collocazione deve rimanere all'interno della griglia, non deve sovrapporsi alle altre navi, non deve condividere alcuna cella con la posizione precedente e non deve utilizzare celle già coinvolte nei colpi effettuati durante la partita. È inoltre necessario mantenere i danni già subiti dalla nave, evitando che il movimento possa ripristinarne lo stato.

La natura casuale dell'evento introduce infine un problema per il testing: utilizzando direttamente una sorgente casuale non controllabile, i test produrrebbero risultati differenti tra un'esecuzione e l'altra.

- Soluzione

L'operazione è stata introdotta nel contratto `Game` tramite il metodo `triggerRandomEvent`. L'utilizzo di `Optional` permette di rappresentare anche il caso in cui nessuna nave possa essere spostata in una posizione valida.

In `GameImpl` l'evento può essere attivato solamente durante la fase `IN_PROGRESS`; negli altri stati viene prodotta la violazione `GAME_NOT_RUNNING`. Viene scelto casualmente uno dei due giocatori come primo candidato e viene richiesto alla sua `Board` di tentare lo spostamento di una nave. Se nessuna nave del primo giocatore può essere spostata, viene effettuato un secondo tentativo sulla griglia dell'avversario.

La logica specifica dello spostamento è stata inserita nell'interfaccia `Board` attraverso `moveRandomUnsunkShip` e implementata in `BoardImpl`. Il metodo costruisce l'insieme delle navi non ancora affondate e ne seleziona casualmente una. Per la nave scelta vengono ricercate tutte le possibili combinazioni di origine e `Rotation` che costituiscono una nuova posizione valida.

La ricerca delle posizioni viene effettuata da `validPlacements`. Per ciascuna possibile collocazione viene costruita temporaneamente una nave candidata tramite `Ship.place`. La posizione viene scartata se condivide anche una sola cella con quella corrente, se una cella esce dai limiti della griglia, se comprende coordinate già bersagliate oppure se si sovrappone a una nave differente. Soltanto le configurazioni che rispettano tutte queste condizioni possono essere utilizzate per il movimento.

Una volta scelta una posizione valida, `applyMove` sostituisce la nave nella struttura interna della `Board`. Le precedenti associazioni tra coordinate e nave vengono rimosse da `occupiedBy` e vengono inserite quelle relative alla nuova

posizione. Anche l'elemento corrispondente della flotta viene sostituito con la nave rilocata.

Lo spostamento effettivo viene realizzato tramite `Ship.relocate`. La nuova nave mantiene lo stesso `ShipId` e lo stesso `ShipType`, mentre le coordinate vengono ricostruite sulla base della nuova origine e rotazione. Lo stato dei danni e della corazza viene trasferito associando le coordinate delle liste ordinate della vecchia e della nuova posizione in modo che il movimento non comporti il recupero delle celle precedentemente danneggiate. La stessa operazione è stata successivamente adeguata a conservare anche lo stato della corazza della `ARMORED_SHIP`.

Il risultato dello spostamento è rappresentato dal record immutabile `ShipMove`, che conserva l'identificativo e il tipo della nave, le coordinate precedenti e successive e il numero di celle danneggiate. `RandomEventResult` associa invece il movimento al `PlayerId` proprietario della griglia interessata dall'evento. In questo modo il risultato dell'evento può essere comunicato agli altri livelli dell'applicazione senza esporre direttamente le strutture modificabili del Model.

Per rendere la meccanica verificabile tramite test automatici, `GameImpl` accetta anche un `RandomGenerator` dall'esterno. Il costruttore normalmente utilizzato continua a impiegare `RandomGenerator.getDefault`, mentre durante i test è possibile fornire una sorgente con valore iniziale noto, ottenendo quindi risultati riproducibili.

I test aggiunti verificano sia il comportamento dello spostamento sia quello dell'evento completo. In particolare, viene controllato che una nave non rimanga nelle stesse celle dopo il movimento, che il numero di danni venga conservato e che `triggerRandomEvent` restituisca le informazioni relative alla nave spostata. È stato inoltre verificato che l'evento venga rifiutato quando la partita non è ancora in corso.

- **Conseguenza**

La gestione dello spostamento rimane principalmente responsabilità della Board, che possiede le informazioni necessarie per verificare la validità delle coordinate e le eventuali sovrapposizioni, mentre `GameImpl` si limita a coordinare l'attivazione dell'evento e la scelta del giocatore interessato.

La rappresentazione del movimento tramite `ShipMove` e del risultato complessivo tramite `RandomEventResult` evita di esporre direttamente gli oggetti interni del Model e rende esplicite le informazioni prodotte dall'evento. Lo stato di una nave viene inoltre preservato durante la rilocazione, impedendo che una meccanica casuale possa cancellare i danni già inflitti durante la partita.

L'iniezione di `RandomGenerator` separa infine la casualità dalla logica dell'evento

e permette di eseguire test deterministici, pur mantenendo un comportamento casuale durante l'utilizzo normale dell'applicazione.

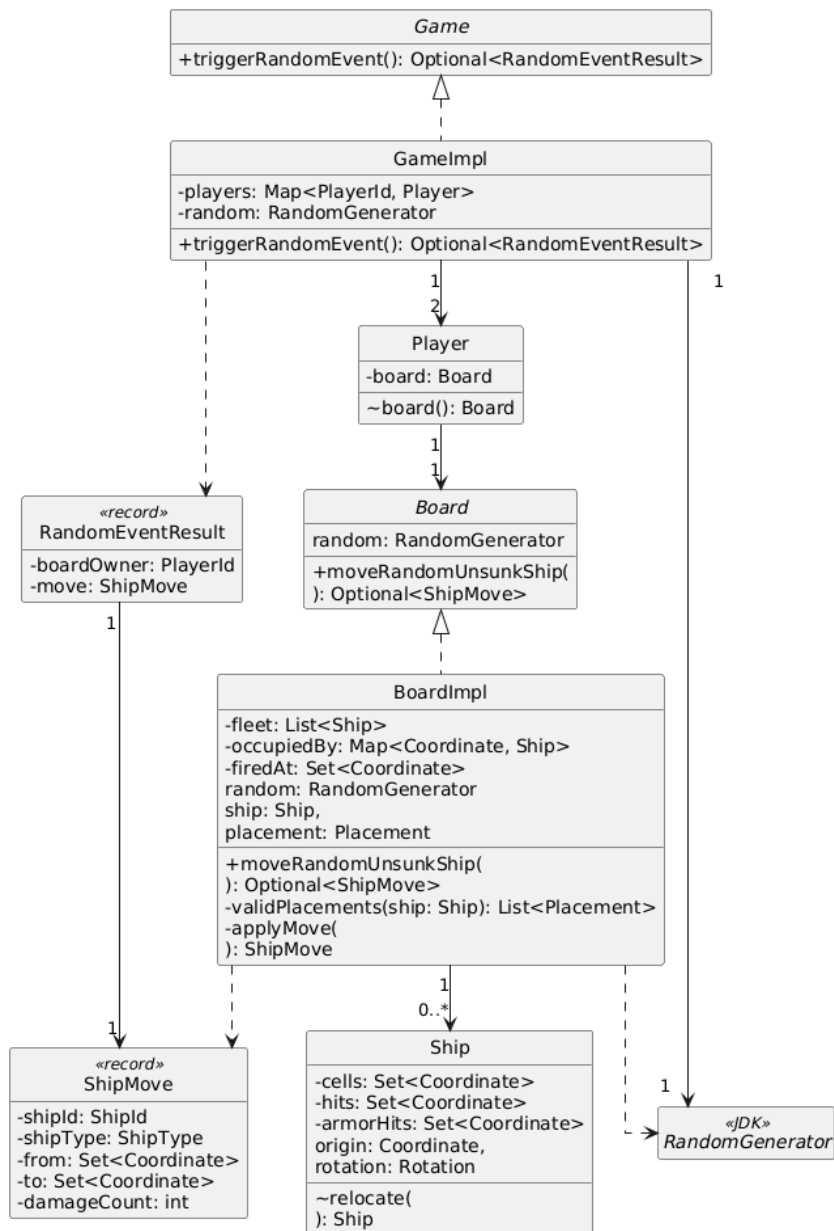


Figura 2.9. Evento casuale e spostamento delle navi

2.2.3 Design dettagliato - Tetyana Volosozhar

2.2.3.1 Gestione delle diverse schermate

- Problema

Il gioco presenta più fasi con contenuti e azioni differenti: configurazione dei nomi, posizionamento delle flotte, combattimento e passaggio del dispositivo tra i due giocatori. Quindi è molto importante evitare una singola schermata sovraccarica e rendere evidente quale operazione è disponibile in ogni momento.

- Soluzione

È stato utilizzato `CardLayout` per associare più pannelli alla stessa area della finestra e di visualizzarne uno alla volta.

In `BattleshipFrame` sono state definite quattro schermate corrispondenti alle card setup, placement, game e privacy.

- `Setup`: consente di inserire i nomi dei due porti, scegliere globalmente se includere la nave corazzata e avviare la fase di posizionamento.
- `Placement`: mostra la griglia del giocatore corrente ed i controlli che servono per scegliere la nave, la rotazione, annullare un posizionamento oppure confermare la flotta completa.
- `Game`: contiene le due griglie del gioco, le informazioni sul turno, le abilità disponibili, i controlli per selezionare il tipo di tiro ed il registro degli eventi.
- `Privacy`: invece è utilizzata durante il passaggio del dispositivo, impedendo così al giocatore successivo di vedere accidentalmente la posizione della flotta dell'avversario.

È importante precisare che il cambio di schermata non è deciso dalla View. Quando l'utente compie un'azione, `BattleshipFrame` inoltra l'informazione al Controller, il quale aggiorna lo stato della partita e chiede alla View di mostrare la schermata appropriata tramite i metodi come `showSetup`, `showPlacement`, `showGame` e `showPrivacy`. La View così si occupa della presentazione, mentre il controller mantiene la responsabilità del flusso dell'applicazione.

`CardLayout` permette di mantenere una sola finestra principale e quindi di separare chiaramente i componenti grafici relativi alle diverse fasi. Questa soluzione riduce la duplicazione del codice, semplifica la gestione dello stato

dell'interfaccia e rende più semplice il ritorno alla schermata iniziale per poter cominciare una nuova partita.

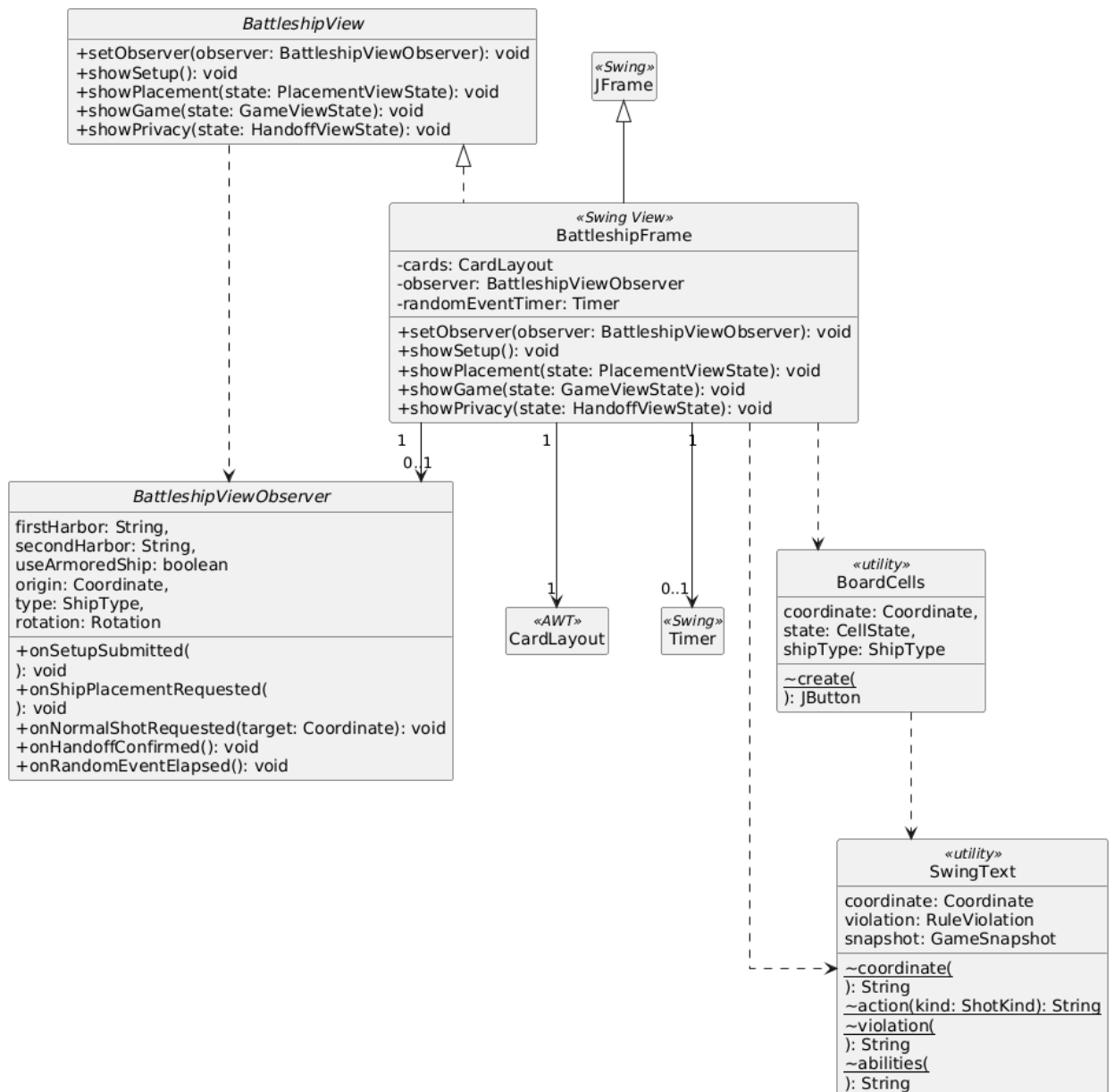


Figura 2.10 Struttura della View Swing

2.2.3.2 Rappresentazione delle griglie

- Problema

Durante il gioco si devono mostrare due griglie; quindi, il giocatore deve poter

vedere la propria griglia e quella dell'avversario. La propria griglia mostra la posizione delle navi, mentre quella dell'avversario deve mostrare solamente le informazioni che il giocatore ha diritto di conoscere; quindi, non può vedere la posizione del concorrente ma deve vedere la griglia per poter colpire le caselle.

- **Soluzione**

La View riceve dal Controller un `BoardSnapshot`, cioè una proiezione immutabile della griglia preparata dal Model. In questo modo la GUI non accede direttamente allo stato interno della partita.

Ogni griglia contiene 100 celle disposte in 10 righe e 10 colonne. La View usa un `GridLayout` 11 x 11: la prima riga contiene le lettere A-J, la prima colonna i numeri 1-10 e le restanti posizioni i pulsanti della tavola di gioco.

`BoardCells` centralizza testo, colore, tooltip e descrizione accessibile di ogni cella in base a `CellState` e all'eventuale `ShipType` visibile. La griglia del proprietario non è usata per sparare; quella dell'avversario diventa interattiva soltanto durante `IN_PROGRESS`. Il primo bersaglio del colpo doppio viene evidenziato fino alla seconda selezione.

La stessa factory grafica garantisce una codifica coerente per acqua, celle ignote, navi, colpi, errori e affondamenti. Lo snapshot dell'avversario non contiene la posizione delle navi non ancora visibili; quindi, la View non può rivelarle accidentalmente.

- **Conseguenze**

Questa soluzione mantiene separati Model e View. La GUI prende solo le informazioni contenute nello snapshot e non può modificare direttamente la griglia o le navi.

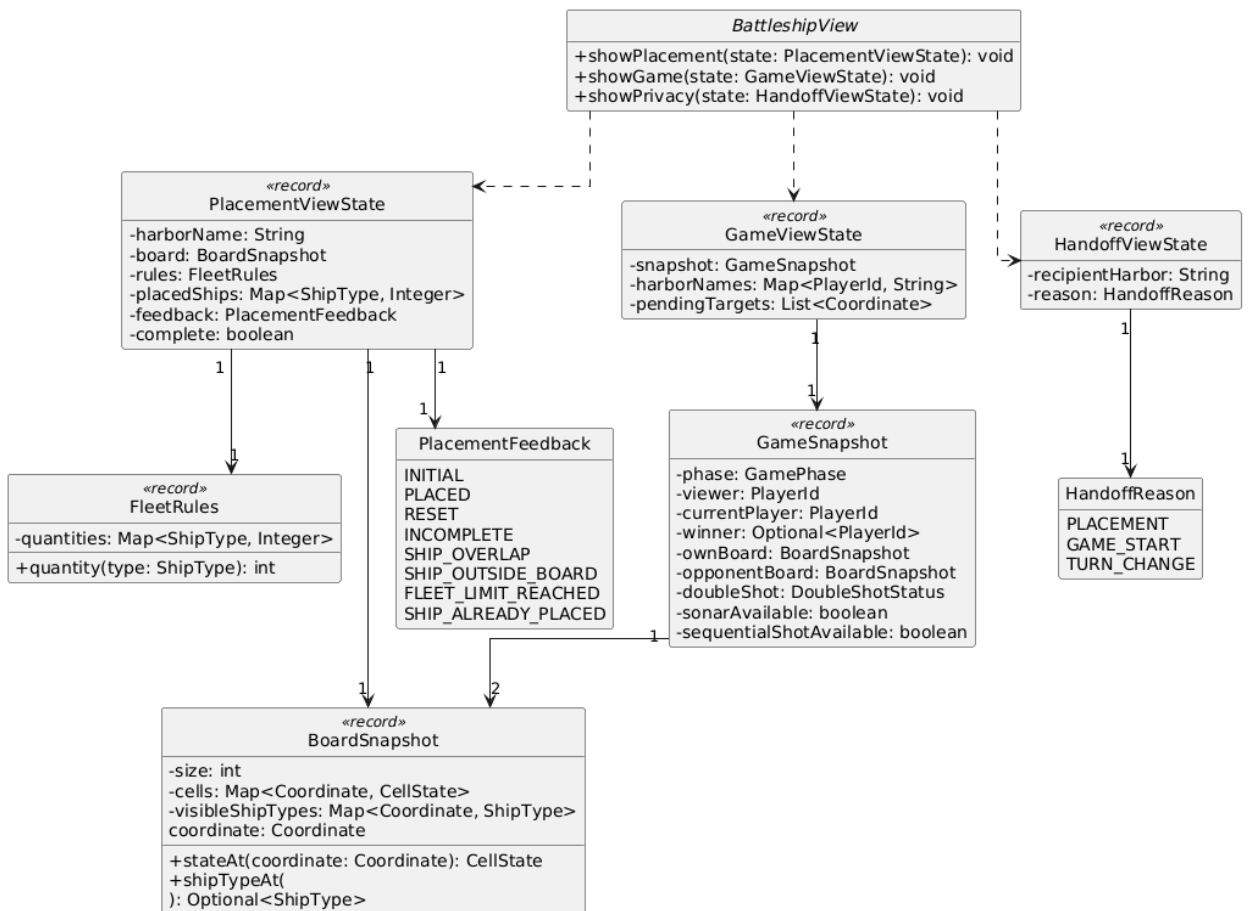


Figura 2.11. Oggetti immutabili usati per aggiornare GUI.

2.2.3.3 - Gestione del posizionamento

- Problema

Prima di iniziare la partita, i giocatori devono posizionare correttamente la propria flotta sulla griglia. L'interfaccia deve quindi permettere di scegliere quale nave collocare, indicarne l'orientamento e segnalare eventuali configurazioni non valide.

- Soluzione

Nella schermata di posizionamento il giocatore seleziona `ShipType` e `Rotation` tramite due menu e fa clic sulla cella di origine. La View comunica l'azione attraverso `BattleshipViewObserver`; il Controller crea la richiesta di dominio e `BoardImpl` verifica limiti, sovrapposizioni e quantità ammesse. La View riceve poi un nuovo `PlacementViewState` e aggiorna griglia, progresso e messaggio di feedback.

Il pulsante "Reset fleet" permette di cancellare il posizionamento corrente e

ricominciare tutto da capo. Il pulsante “Confirm fleet” invece può essere usato quando il Model comunica che tutte le navi sono state posizionate.

La View non contiene le regole per stabilire se una nave è collocata correttamente. Questo controllo appartiene al Model, il Controller invece si occupa di collegare l’input dell’utente con la logica del gioco e di riportare alla View l’eventuale errore.

2.2.3.4 - Selezione delle modalità di tiro

- Problema

Durante la fase di combattimento, il giocatore può effettuare azioni diverse (colpo normale, colpo doppio o colpo sequenziale). L’interfaccia deve permettere di scegliere l’azione corretta.

- Soluzione

La enum `ActionMode` rappresenta le modalità di tiro disponibili nella GUI ed il giocatore, usando il menu a tendina le può selezionare.

Quando viene scelta la modalità `Sequential shot`, viene abilitato anche il menu per selezionare la direzione: destra, sinistra, sopra e sotto. Le direzioni sono rappresentate dalla enum `ShotDirection`, che associa a ciascuna scelta lo spostamento e lo può applicare alla riga o alla colonna della griglia.

Nel caso del colpo normale, è sufficiente selezionare una cella della griglia avversaria.

Nel colpo doppio, il Controller memorizza il primo bersaglio e la View lo evidenzia; il secondo clic completa un'unica azione atomica con due coordinate distinte. L'abilità si carica dopo tre colpi `NORMAL` riusciti, torna a zero dopo l'uso e può essere ricaricata più volte. Un tentativo prematuro viene rifiutato senza modificare turno, griglia o carica.

Nel colpo sequenziale, il Controller usa la cella iniziale e la direzione scelta per calcolare le tre coordinate coinvolte. Prima di inviarle l’azione al Model, verifica che tutte le coordinate rimangano all’interno dei limiti della griglia. Se una parte di tiro esce dalla tavola di gioco, l’azione viene bloccata e la View mostra un messaggio di errore.

2.2.3.5 - Privacy, feedback e aggiornamento dell’interfaccia

- Problema

La partita è pensata per due giocatori che usano lo stesso dispositivo; quindi, è necessario impedire che un giocatore possa vedere la flotta dell’altro durante il

cambio di turno e fornire informazioni chiare durante ogni azione.

- **Soluzione**

Quando il turno passa all'altro giocatore, il Controller richiede alla View di mostrare una schermata di privacy. Questa schermata comunica che il dispositivo deve essere passato al giocatore successivo e nasconde temporaneamente la schermata di gioco.

Dopo ogni azione, il Controller riceve dal Model il risultato e richiede un nuovo aggiornamento alla View. La GUI aggiorna le griglie, il testo relativo al turno e alle abilità disponibili e aggiorna il registro della partita. Il registro è un'area di testo non modificabile, nella quale vengono aggiunti gli esiti dei colpi e gli eventuali eventi che si verificano durante la partita.

La View dispone anche di finestre di dialogo per mostrare informazioni e avvisi. Queste informazioni vengono usate quando il giocatore seleziona una coordinata non valida oppure quando prova ad usare un'abilità già consumata oppure quando finisce la partita. Infatti, a fine partita viene mostrato un messaggio con il nome del vincitore.

È presente anche un segnale sonoro tramite "Beep" che fornisce un riscontro immediato al giocatore.

La View gestisce un `javax.swing.Timer` che notifica periodicamente l'Observer. Il timer non modifica il Model: il Controller decide se richiedere `triggerRandomEvent`. Durante le schermate di passaggio del dispositivo il timer viene fermato, evitando cambiamenti nascosti mentre entrambi i tabelloni sono coperti.

Il Controller richiede il feedback sonoro soltanto quando almeno uno `ShotResult` ha `isHit` uguale a `true`, quindi per `HIT` o `SUNK`. `MISS` e `ARMOR_ABSORBED` non producono il segnale. La View usa il segnale acustico di sistema senza modificare lo stato della partita.

L'effetto sonoro è gestito dalla View tramite il segnale acustico di sistema. Non modifica lo stato della partita e non influisce sul risultato dell'azione, ma rende più immediata la comunicazione dell'esito al giocatore.

Questa organizzazione mantiene separati presentazione, coordinamento e logica di dominio. La View non decide il risultato delle azioni, ma visualizza lo stato ricevuto dal Controller. Il Controller coordina il flusso, mentre il Model conserva le regole della partita

- **Conseguenza**

La schermata di passaggio impedisce l'esposizione involontaria delle flotte e mantiene il timer separato dalle regole del Model. Il feedback grafico e sonoro

migliora la comprensibilità senza modificare lo stato della partita.

Capitolo 3 Sviluppo

3.1 Testing automatizzato

La suite automatizzata comprende 35 metodi annotati con `@Test` ed è organizzata per responsabilità. `ModelBasicsTest` verifica gli invarianti di `Coordinate` e l'alternanza fra `PLAYER1` e `PLAYER2`. `FleetRulesTest` controlla le quantità delle configurazioni, le copie difensive e il rifiuto dei valori invalidi. `BoardImplTest` copre colpi, piazzamento, visibilità, sonar, esposizione delle regole di flotta e le quattro rotazioni distinte della `RECON`.

`GameImplTest` verifica gestione del turno, colpo doppio, sonar, vittoria, assorbimento della corazza, compatibilità delle `FleetRules` e presenza esplicita di tutte le strategie. `SpecialMechanicsTest` esercita la corazza per singola sezione, la geometria del tiro sequenziale, il movimento casuale, la conservazione dei danni e della corazza, gli snapshot e l'evento casuale.

`CompleteGameSmokeTest` simula una partita completa del Model fino allo stato `FINISHED`. `BattleshipAppSmokeTest` verifica che la finestra possa essere costruita e chiusa. `FleetFactory` è collocata esclusivamente nel codice di test e permette di preparare flotte complete senza introdurre dipendenze dal testing nel codice di produzione.

I test relativi alla casualità utilizzano sorgenti con seed noti, così da ottenere risultati riproducibili. La verifica viene considerata superata soltanto quando test automatici, Checkstyle, PMD e gli altri analizzatori configurati terminano senza errori.

3.2 Note di sviluppo

3.2.1 Descrizione di alcune funzionalità utilizzate nella realizzazione della GUI - Tetyana Volosozhar

- **Lambda expression e gestione degli eventi.** Le lambda expression vengono usate per collegare i componenti grafici alle azioni corrispondenti. Per esempio, i pulsanti e le celle della griglia utilizzano degli `ActionListener` per richiamare i metodi del Controller o della View. Alcuni esempi si trovano alle righe **256-257**, **288-291**, **336-337**, **348-352**, **399** e **438** di `BattleshipFrame.java`.

Permalink: [BattleshipFrame.java, righe 256-257](#)

- **CardLayout.** È stato usato per gestire le diverse schermate dell'applicazione. I metodi che mostrano le diverse schermate sono invece alle righe **122-150** e **154-169**. Le schermate disponibili sono quella iniziale, quella del posizionamento, quella della partita e quella per il passaggio del dispositivo.
Permalink: [BattleshipFrame.java, righe 74-75, 107-110](#)
- **Timer di Swing.** Il `javax.swing.Timer` viene usato per notificare periodicamente il Controller della possibilità di verificare un evento casuale. Il timer viene creato e avviato alle righe **198-207**, mentre viene fermato alle righe **210-215**.
Permalink: [BattleshipFrame.java, righe 198-207](#)
- **Layout manager di Swing.** Sono stati usati diversi layout per organizzare la finestra, i controlli e le griglie. La schermata iniziale usa `GridBagLayout` alle righe **217-235**; la schermata di posizionamento usa `BorderLayout`, `BoxLayout` e `BorderFactory` alle righe **266-301**; la schermata di gioco usa `BorderLayout`, `BoxLayout` e `JSplitPane` alle righe **304-355**. Le griglie vengono costruite con `GridLayout` alle righe **384-405** e **416-450**.
Permalink: [BattleshipFrame.java, costruzione dei pannelli](#)
- **Uso di Optional nella lettura dello snapshot.** Quando viene costruita una cella, la View recupera il tipo di nave eventualmente presente usando `shipTypeAt(...).orElse(null)`. In questo modo il caso in cui non ci sia una nave viene gestito senza utilizzare una stringa speciale o un valore numerico convenzionale. Questo utilizzo è visibile alle righe **395-398** e **433-436**.
Permalink: [BattleshipFrame.java, righe 433-436](#)

La GUI utilizza le librerie Swing e AWT già presenti nel JDK. Non sono state utilizzate librerie grafiche esterne.

3.2.2 Descrizione di alcune funzionalità utilizzate nella realizzazione di dinamiche, colpi ed eventi speciali

- Angelica Sartori

- **Uso di lambda expressions**
Utilizzate in `Ship.relocate` per definire il criterio di ordinamento delle coordinate durante il trasferimento dello stato della nave corazzata.
Permalink: [Ship.java - righe 203-209](#)
- **Uso della Stream API**
Utilizzata in `SequentialShotStrategy` per ordinare le coordinate e verificare tramite `map`, `distinct` e `count` che i tre bersagli appartengano

alla stessa riga o colonna.

Permalink: [SequentialShotStrategy.java - righe 35-45](#)

- **Uso di Optional**

Utilizzato in GameImpl.triggerRandomEvent per rappresentare esplicitamente la possibilità che l'evento casuale non produca alcuno spostamento valido, evitando l'utilizzo di null.

Permalink: [GameImpl.java - righe 205-217](#)

- **Uso della libreria JUnit per il testing automatico**

JUnit è stato utilizzato nei test delle nuove meccaniche, ad esempio tramite assertThrows con una lambda per verificare il rifiuto di una configurazione non valida del tiro sequenziale.

Permalink: [SpecialMechanicsTest.java - righe 83-88](#)

3.2.3 Descrizione di alcune funzionalità avanzate nella realizzazione del Model - Marco Penolazzi

- **Record e costruttore compatto per un value object immutabile.**

Coordinate è stato realizzato come record; il costruttore compatto verifica che riga e colonna siano non negative, impedendo la costruzione di coordinate non valide.

Permalink: [Coordinate.java, righe 9-20.](#)

- **Stream API e lambda expression per interrogare una collezione.**

In BoardImpl.placeShip, anyMatch verifica che l'identificatore della nave non sia già presente, mentre filter e count calcolano quante navi dello stesso tipo sono state posizionate senza esporre la collezione interna.

Permalink: [BoardImpl.java, righe 47-52.](#)

- **Optional e copia difensiva per rappresentare uno stato immutabile.**

TurnResult usa Optional per rappresentare esplicitamente l'eventuale assenza del prossimo giocatore e del vincitore; il costruttore compatto usa List.copyOf per non conservare una lista modificabile dal chiamante.

Permalink: [TurnResult.java, righe 16-31.](#)

Capitolo 4 Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Autovalutazione - Tetyana Volosozhar

Il mio contributo principale al progetto è stato lo sviluppo della GUI e del Controller. Mi sono occupata di creare le schermate dell'applicazione, visualizzare le griglie, gestire i pulsanti e i menu e collegare le azioni dell'utente alle operazioni del gioco.

La parte più difficile è stata realizzare l'interfaccia grafica e coordinare il mio lavoro con quello degli altri componenti del gruppo. La GUI doveva infatti utilizzare correttamente le informazioni fornite dal Model e rispettare le regole implementate dagli altri membri. È stato quindi necessario confrontarsi spesso per decidere come gestire i dati e come aggiornare l'interfaccia dopo ogni azione.

Un aspetto positivo del mio lavoro è stato riuscire a separare la parte grafica dalla logica del gioco. La View mostra le informazioni e raccoglie le azioni dell'utente, mentre il Controller comunica con il Model. In questo modo la GUI non modifica direttamente lo stato della partita.

La GUI è verificata mediante uno smoke test che controlla sullo Swing Event Dispatch Thread la costruzione e la chiusura della finestra. Non sono invece presenti test automatici completi delle interazioni dell'utente con tutti i componenti Swing.

In futuro sarebbe possibile migliorare la grafica delle navi e delle celle, aggiungere test automatici per la GUI e tradurre i messaggi in più lingue. Si potrebbe anche sostituire Swing con un'altra tecnologia grafica, mantenendo separata la logica del Model dall'interfaccia.

Questa esperienza mi ha permesso di comprendere meglio il funzionamento del pattern MVC e l'importanza di coordinare le diverse parti di un progetto software. Ho inoltre imparato che la realizzazione della GUI non consiste solo nel creare schermate, ma richiede un collegamento preciso con il Controller e con i dati prodotti dal Model.

4.1.2 Autovalutazione - Angelica Sartori

Il mio contributo al progetto si è concentrato principalmente sul Model e, in particolare, sull'introduzione e sull'integrazione di alcune meccaniche speciali. Ho lavorato sulla gestione della nave corazzata, sull'introduzione del tiro sequenziale e sulla realizzazione dell'evento casuale che permette lo

spostamento delle navi. Ho inoltre realizzato test automatici relativi a queste funzionalità e una utility di supporto per facilitare la costruzione delle flotte durante il testing.

Tra i principali punti di forza del mio lavoro considero soprattutto il tentativo di integrare le nuove funzionalità nelle strutture già presenti nel progetto, evitando di introdurre logiche completamente separate. Il tiro sequenziale, ad esempio, è stato inserito sfruttando l'astrazione già esistente per le strategie di tiro, mentre la gestione della nave corazzata è stata progressivamente raffinata fino a rappresentare la protezione separatamente per ogni cella della nave. Anche l'evento casuale è stato progettato in modo da preservare lo stato già raggiunto dalle navi, evitando che uno spostamento possa cancellare i danni subiti in precedenza.

Un altro aspetto positivo è stato il ricorso al testing automatico tramite JUnit. Le principali meccaniche introdotte sono state verificate con test specifici e, oltre a questi, è stato realizzato uno smoke test per controllare che le diverse componenti del Model potessero collaborare correttamente durante una partita completa. Per le funzionalità che dipendono dalla casualità è stata utilizzata una sorgente controllabile, così da rendere i test riproducibili.

Dal punto di vista dei limiti, alcune funzionalità sono state definite e corrette nelle fasi finali dello sviluppo. La prima implementazione della nave corazzata, ad esempio, utilizzava una protezione unica per tutta la nave e si è poi reso necessario modificarla per ottenere un comportamento più coerente con le regole previste. Inoltre, l'introduzione delle meccaniche speciali ha aumentato il numero di casi che il Model deve gestire contemporaneamente, rendendo più complessa l'interazione tra colpi, stato delle navi e regole di gioco. Anche se i test automatici coprono le principali funzionalità sviluppate, non verificano tutte le possibili combinazioni tra le diverse meccaniche speciali.

In un'eventuale prosecuzione del progetto sarebbe utile ampliare la copertura dei test automatici, soprattutto per quanto riguarda l'interazione tra più meccaniche speciali, e rendere maggiormente configurabili le regole relative agli eventi e alle abilità, così da facilitare l'aggiunta di nuove modalità di gioco.

4.1.3 Autovalutazione - Marco Penolazzi

Il mio contributo principale ha riguardato il nucleo deterministico del Model: value object e violazioni, rappresentazione delle navi, Board e snapshot, strategie di tiro, carica del colpo doppio e coordinamento della partita in GameImpl, insieme ai relativi test. Considero un punto di forza la separazione fra stato mutabile interno e risultati immutabili, che permette di verificare le regole senza avviare Swing. Anche l'uso di Strategy, Template Method e Decorator è rimasto collegato a problemi concreti e non è stato aggiunto soltanto per mostrare un pattern.

La difficoltà maggiore è stata mantenere coerenti regole introdotte in momenti diversi, in particolare flotta opzionale, corazza, tiro sequenziale ed eventi casuali. Alcune classi, soprattutto BoardImpl e GameImpl, hanno una responsabilità ampia e potrebbero essere ulteriormente suddivise se il numero di meccaniche aumentasse; nella dimensione attuale del progetto ho preferito non introdurre astrazioni prive di un bisogno effettivo. Un ulteriore sviluppo potrebbe aggiungere salvataggio della partita e una seconda implementazione della View, mantenendo invariato il Model. Ritengo che il risultato raggiunto sia solido nella correttezza delle regole e nella testabilità, mentre la documentazione e la rifinitura grafica hanno richiesto una fase finale di allineamento con il lavoro degli altri membri.

Appendice A

Guida utente

Avviare il gioco con `java -jar OOP25-battaglia-navale-all.jar`. Nella schermata iniziale inserire i nomi dei due porti e scegliere se includere la nave corazzata: l'opzione vale per entrambi i giocatori. Se disattivata, ogni flotta contiene otto navi; se attivata ne contiene nove.

Il primo giocatore seleziona un tipo di nave, sceglie la rotazione 0°, 90°, 180° o 270° e fa clic sulla cella di origine. Il pannello mostra quante navi sono state piazzate e quante ne restano. Le navi non possono uscire dalla griglia o sovrapporsi. `Reset fleet` cancella il piazzamento corrente dopo una conferma; `Confirm fleet` si abilita soltanto a flotta completa. Terminato il primo piazzamento, una schermata chiede di passare il dispositivo al secondo giocatore senza mostrare il tabellone precedente.

Durante la battaglia la griglia a sinistra mostra la propria flotta, quella a destra il campo avversario. Le colonne sono indicate da A a J e le righe da 1 a 10. I simboli principali sono • per un colpo mancato, ✕ per una sezione colpita e O per una nave affondata; le lettere A, B, C, S, D, R e X identificano le proprie navi. I tooltip riportano anche la coordinata.

Normal shot richiede una cella. Double shot richiede due celle differenti ed è disponibile dopo tre colpi normali riusciti; una volta usato riparte da zero e può essere ricaricato. Sequential shot seleziona tre celle consecutive nella direzione scelta ed è disponibile una sola volta per giocatore. Sonar analizza un'area 3×3, non rileva il sottomarino invisibile, è disponibile una sola volta e termina il turno. Il primo impatto su ciascuna sezione della nave corazzata viene assorbito e non produce danno.

Un HIT o SUNK mantiene il turno; MISS e ARMOR_ABSORBED lo passano. Quando il turno cambia compare una schermata di privacy: consegnare il dispositivo al porto indicato e premere Continue soltanto quando il nuovo giocatore è pronto. Gli eventi casuali possono spostare una nave non affondata conservandone i danni. La partita termina quando tutte le navi di una flotta sono affondate.

Appendice B

Esercitazioni di laboratorio

B.0.1 marco.penolazzi@studio.unibo.it

- Laboratorio 06:
<https://virtuale.unibo.it/mod/forum/discuss.php?d=206731#p284038>
- Laboratorio 07:
<https://virtuale.unibo.it/mod/forum/discuss.php?d=207193#p284946>
- Laboratorio 08:
<https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286051>
- Laboratorio 09:
<https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p286857>
- Laboratorio 10:
<https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288177>
- Laboratorio 11:
<https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289415>
- Laboratorio 12:
<https://virtuale.unibo.it/mod/forum/discuss.php?d=211539#p290379>